



SIGNAL SERIES • START HERE

START HERE

Foundations, Git and CI — the ground everything stands on.

By **Nabawy**

AUTHOR & COMPILER

Foundations

Source Control

Integration

Table of Contents

03	The Map — The Whole Route	03	03B	What CI/CD Actually Is	03B
04	Foundations	04	05	Continuous Integration	05
06	CI vs CD · Delivery vs Deployment	06	07	Pipeline & Status	07
08	Environments & Flow	08	09	Console	09
10	Artifact & Gate	10	11	Failure & Recovery	11
12	Check & Map	12	13	Source Control	13
14	Flow & Commands	14	15	Models & Discipline	15
16	Check & Handoff	16	17	Integration	17
18	Gates	18	19	Handoff	19
20	Quick Revision — Terms Table	20	21	Concept Questions — Distilled Answers	21
22	Command Cheat Sheet	22	23	Self-Quiz + Answer Key	23

▪ **How to use this book** legend

NOTE · context KEY · must retain WATCH OUT · gotcha VERIFIED · check

NOTE Read cover to cover once, then use the Contents to jump. Every chapter ends with a EXAM box — if you can answer it cold, move on.

The Map — The Whole Route

Three chapters, one line: [Foundations](#) draws the pipeline every merge travels; [Source Control](#) builds the gate that guards it; [Integration](#) turns the gate into daily discipline.

FOUNDATIONS_{lifecycle}



SOURCE CONTROL_{gate}



INTEGRATION_{discipline}

commit → verify → merge → repeat

KEY IDEA Small merges, verified automatically, fixed immediately — that discipline feeds every pipeline downstream.

What CI/CD Actually Is

CI/CD is automation plus discipline: every commit verified by machines, every release decided by policy. One line: commit → CI → artifact → CD → observe.



Practice	Question it answers
Continuous Integration	Does this change work — build, test, scan green?
Continuous Delivery	Is it releasable — staged auto, prod by approval?
Continuous Deployment	Did it ship — prod automatic, rollback armed?
GitOps	Is reality matching git — pull, drift-corrected?

KEY IDEA CI answers does-it-work; Delivery keeps it releasable; Deployment ships it.

EXAM Name which gate each change passed: CI gate on merge, approval gate on release, sync gate on deploy.

Next: [Foundations](#)

Foundations

A pipeline is not magic.

It is a *deterministic sequence of machines making decisions about your code* — from the moment you push to the moment users feel the change.

1 / THE CI/CD LIFECYCLE



Every chapter of this manual lives somewhere on this line. Feedback flows backward: deploy → observe → next commit.

2 / HOW TO READ THIS MANUAL

Each concept answers **WHAT · WHY · WHEN · HOW · WHAT CAN FAIL**. Green means passed, red means failed, amber means a human must decide, purple means artifact. Color always pairs with a symbol and a label — the book survives grayscale printing.

KEY IDEA DevOps is the culture of shared ownership; CI/CD is its automation — small merges verified on every push, releasable always, deployed by decision.

Continuous Integration

3 / WHAT IS CONTINUOUS INTEGRATION

 **Developers merge small changes into shared main frequently; every merge triggers an automated build + test + scan.** CI answers one question: *does this change work?*

WHEN — on every push and every pull request. **HOW** — a pipeline job checks out the exact commit, builds it in a clean environment, and reports green or red back to the branch in minutes.

WHY THIS EXISTS

Without automated integration, a broken change travels for days before anyone discovers it. CI moves that discovery to within minutes of the commit — when the author still remembers what they did.

4 / WHAT HAPPENS WITHOUT CI/CD

- **Long-lived branches** → **merge hell**. Weeks of divergent work collide; untangling takes days.
- **"Works on my machine" releases**. Testing one thing, shipping another — every deploy a gamble.
- **Manual testing** → **bugs reach users**. Humans skip steps under pressure.
- **Manual deploys** → **release fear**. Rare, big, traumatic events nobody runs on Friday.
- **No feedback loop**. The same bug class ships again and again.

Integration pain is paid **minutes per commit** with CI/CD, or **weeks per release** without it.

Threads: [Trunk Discipline](#)

CI vs CD • Delivery vs Deployment

5 / CI VS DELIVERY VS DEPLOYMENT

	CI	Continuous Delivery	Continuous Deployment
Question	Does it work?	Could we ship it?	— (ships itself)
Trigger	Commit / merge	Validated artifact	Validated artifact
Human gate	No	Yes — approval	No
Output	Verified code	Releasable artifact	Production deployment

DO YOU DEPLOY TO PRODUCTION AUTOMATICALLY?

YES → CONTINUOUS DEPLOYMENT

Every passing change ships. Requires progressive delivery + instant rollback.

NO → CONTINUOUS DELIVERY

Every passing change is releasable. A human or policy decides when.

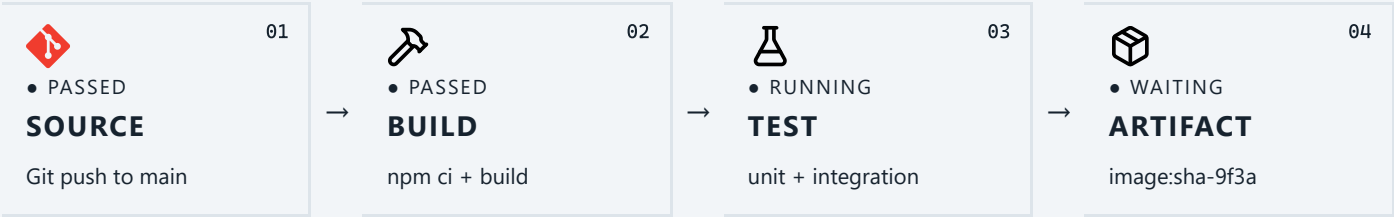
GOTCHA

A green build does not mean the application works in production. It means only the checks inside that pipeline passed.

Threads: [Approval Gates](#)

Pipeline & Status

6 / ANATOMY OF A PIPELINE



Stages are phases, jobs are units of work inside them, steps are single operations, runners are the machines. Progression reads left to right; a red stage blocks everything downstream.

Term	Meaning
Pipeline	The automated workflow
Stage	Logical phase (build, test, deploy)
Job	Unit of executable work inside a stage
Step	Single operation inside a job
Runner / Agent	Execution environment running the work

7 / THE STATUS SYSTEM

• RUNNING

• PASSED

• FAILED

• BLOCKED

• WAITING

• SKIPPED

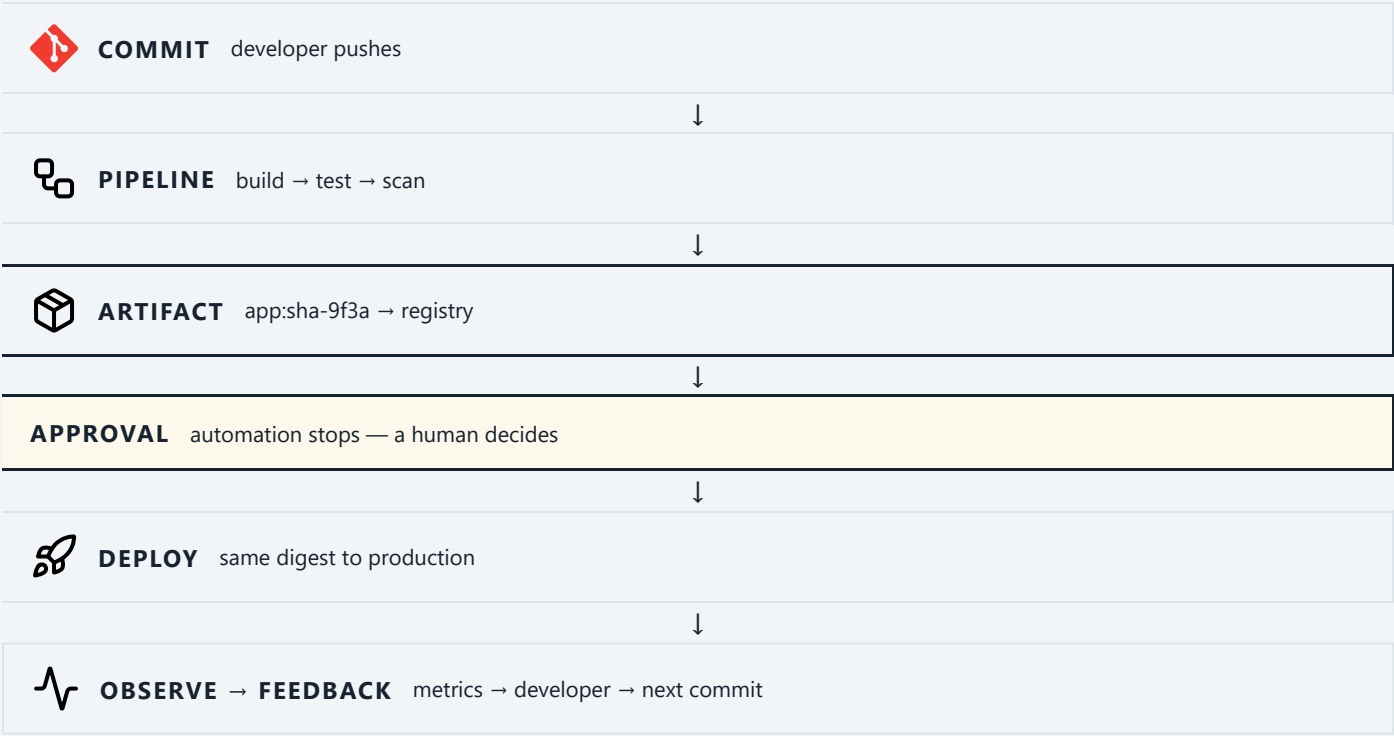
Threads: [Controllers & Agents](#)

Environments & Flow

8 / ENVIRONMENT LANES: ONE ARTIFACT, THREE TRACKS

DEVELOPMENT	BUILD → TEST
push → build ✓ → unit tests ✓ → image staged	
STAGING	DEPLOY → VERIFY
deploy sha-9f3a → smoke ✓ → scan ✓ → awaiting approval	
PRODUCTION	APPROVE → DEPLOY
◦ approval gate → deploy same digest → observe	

9 / THE DEPLOYMENT LOOP



Threads: [Artifact Digest](#) · [Approval Gates](#)

Console

10 / FROM PUSH TO PIPELINE

```
$ git push origin main

→ pipeline triggered · commit 9f3a2c1

[01] BUILD      PASSED    1m 12s
[02] TEST       RUNNING   · 214/300
[03] SCAN       WAITING
[04] ARTIFACT   WAITING
```

11 / COMMAND AND RESULT

COMMAND

```
$ docker build -t registry/app:sha-9f3a .
$ docker push registry/app:sha-9f3a
```

RESULT

```
Successfully built 7f91c2
Successfully tagged sha-9f3a
Pushed digest sha256:4d...e9
```

WHY IT MATTERS the pushed digest is the frozen, promotable unit. Staging and production deploy this exact hash; nothing is ever rebuilt between environments.

Threads: [Artifact Digest](#)

Artifact & Gate

12 / THE ARTIFACT IDENTITY



SOURCE CODE repo @ 9f3a2c1



BUILD npm ci → bundle



ARTIFACT · **app:sha-9f3a** Docker image · JAR · binary · Helm chart



REGISTRY stored, versioned, distributable



DEPLOY same digest everywhere

13 / THE APPROVAL GATE

AUTOMATED build ✓ · test ✓ · scan ✓ · artifact ✓ → **APPROVAL — a human decision is required** → **PRODUCTION**

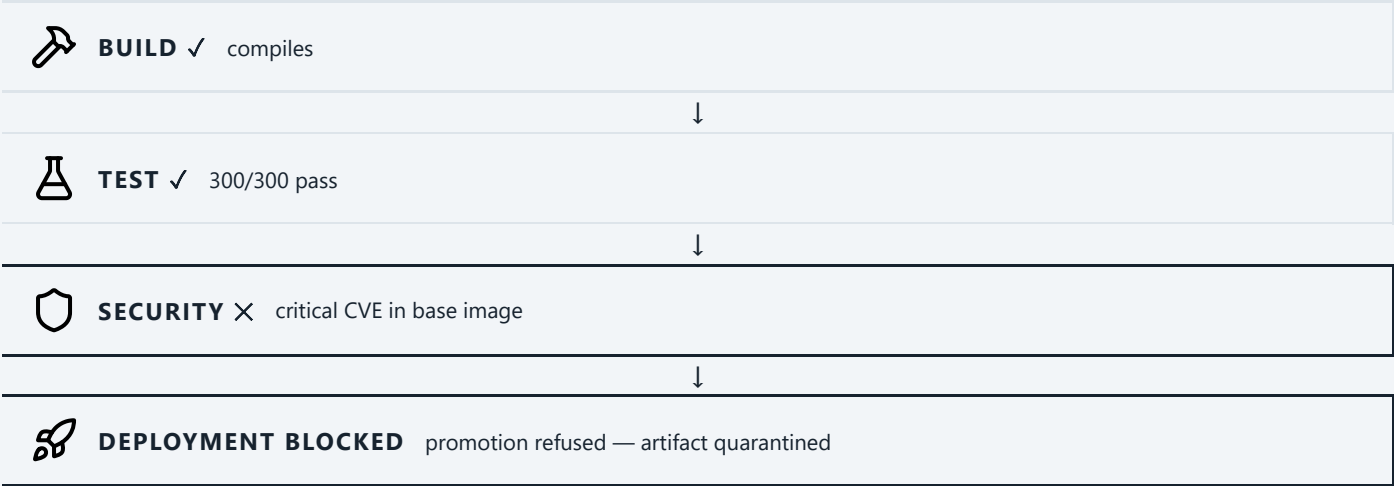
CONTEXT — PRODUCER & CONSUMER

The build pipeline **produces** the artifact; the deployment pipeline **consumes** it; the registry **stores** it. Lifecycle: build → store → promote → deploy. Anything that breaks this chain — a rebuild, a mutable tag, a skipped gate — voids every check upstream.

Threads: [Artifact Digest](#) · [Approval Gates](#)

Failure & Recovery

14 / FAILURE AS A STATE TRANSITION



RECOVERY LOOP
FAILURE → LOGS → ROOT CAUSE → FIX → NEW COMMIT → PIPELINE. Every incident must land a new check, or it will recur.

GOTCHA
Never bypass a red gate with an admin override "just this once". The override becomes the process.

Threads: [Artifact Digest](#)

Check & Map

15 / INTERVIEW CHECK

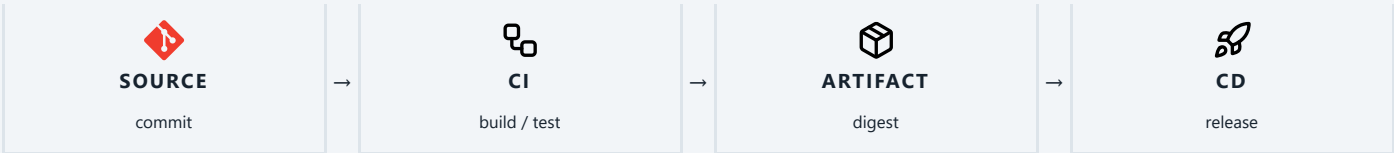
Q1 Delivery vs Deployment?

Delivery keeps software production-ready behind an approval; Deployment releases validated changes automatically.

Q2 Why must staging and production share one artifact digest?

Otherwise you tested one binary and shipped another — verification is void.

16 / THE CI/CD SYSTEM MAP



EXAM CI/CD in one breath: every merge to **main** triggers a pipeline that builds, tests and scans in minutes; stages run jobs on runners and produce exactly one **immutable artifact digest**; **Continuous Delivery** keeps every change releasable behind an approval, **Continuous Deployment** ships it; staging and production deploy the **same digest**; a red gate is never bypassed.

Next: Source Control

Threads: [Artifact Digest](#) · [Approval Gates](#)

Source Control

Picking up from *Foundations*, where a Git push triggered the lifecycle — we now examine the machinery of that trigger: *branches isolate work, pull requests gate it, triggers fire pipelines*.

01 / WHAT EACH PIECE IS

 **Git** — distributed version control; every commit is a snapshot with author, message, parent.

 **Branching** — an independent line of work (feature/login) forked from main.

 **Pull Request** — a merge proposal combining human review with automated status checks.

 **CI Trigger** — the event that starts a pipeline: PR opened/updated, push to main, schedule, manual.

02 / WHY THE GATE EXISTS

Direct pushes to main bypass review and verification. The branch → PR → CI → merge flow makes every change **Reviewed, Tested, and Traceable** before it can break the pipeline.

Threads: [Approval Gates](#)

Flow & Commands

03 / HOW THE FLOW EXECUTES



BRANCH small logical commits



PULL REQUEST trigger fires



CHECKS build · test · scan post back



MERGE green + approved only

04 / COMMANDS

```
$ git checkout -b feature/login
$ git add -A && git commit -m "feat(auth): add login validation"
$ git push -u origin feature/login
```

```
# then open the PR – CI trigger fires automatically
```

```
# .github/workflows trigger: CI on every PR + push to main
on:
  pull_request:
  push:
    branches: [main]
```

Models & Discipline

05 / BRANCHING MODELS

Model	Rule	Best for
GitHub Flow	Branch → PR → main → deploy	CD teams (this series' default)
Git Flow	develop + release/* + hotfix/*	Versioned releases
Trunk-Based	Tiny branches, merge daily	High-velocity CI

06 / FAILURE MODES & BEST PRACTICES

- Long-lived branches → merge hell; fix with trunk-based discipline.
- Red checks ignored; fix with required status checks.
- Merge-queue races; fix with merge queues, linear history.

RULES one logical change per PR (< ~400 lines) · green CI + 1 approval before merge · conventional commits (feat:, fix:) so changelogs automate.

GOTCHA

"Merging equals deploying" is false. Merging makes a change a *candidate*; Continuous Delivery decides deployment.

Threads: [Trunk Discipline](#) · [Approval Gates](#)

Check & Handoff

07 / INTERVIEW CHECK

Q3 Explain the PR as a quality gate.

Human review (design) + automated checks (correctness) + merge policy (green + approved = mergeable).

Q4 Map the trigger types.

Push, pull request, schedule, manual dispatch, tag/release.

08 / HANDOFF

The trigger fires — now something must execute. This workflow continues in **Pipelines, Build, Test & Artifacts**, where the triggered pipeline runs build, test, and scan.

🎓 **EXAM** Branch small → open a PR → CI verifies → merge on **green + approval**. **Trunk-based** is the default: tiny branches, daily merges, flags instead of long branches. **Merge queues** serialize busy mains; **linear history** keeps main bisectable. Merging makes a change a **candidate** — Delivery decides deployment.

Next: Integration

Threads: [Approval Gates](#) · [Trunk Discipline](#)

Integration

Picking up from *Source Control*, where the PR gate was built — CI is *the discipline that makes the gate meaningful*: integrate daily, verify automatically, fix immediately.

01 / THE THREE HABITS



INTEGRATE branches live hours, never weeks



VERIFY every merge runs the full gate



FIX red main = top priority, stop the line

Integration pain grows with branch age: two-day branches merge in minutes, two-month branches in weeks.

Gates

02 / PR GATES & MERGE QUEUES

- **PR gate** — required checks + approvals in branch protection. No green, no merge. No admin overrides.
- **Merge queue** — serializes merges on busy main: rebase onto latest green, re-verify, land. Kills the green-on-branch-red-on-main race.
- **Linear history** — rebase/squash keeps main bisectable: any commit builds and reverts cleanly.

03 / TRUNK DISCIPLINE

RULES tiny branches, daily merges · flags hide unfinished work, not long branches · main always releasable · whoever breaks it fixes it first · measure merges per dev per week.

- Long branches → merge hell (WIP limits + flags).
- Red main normalized → quality collapse (stop-the-line culture).
- Flaky gates → bypassed checks (quarantine flakes, never disable gates).

Q5 CI practice vs CI pipeline?

Discipline (merge small, verify, fix first) vs machinery (the pipeline that enforces it).

DISCIPLINE DETAILS preflight builds validate diffs before landing on main · rotating build master reverts lingering breakage on large teams · ratcheting gates fail only when warnings or duplication increase.

Threads: [Trunk Discipline](#) · [Approval Gates](#)

Handoff

04 / HANDOFF

Small merges, verified automatically, fixed immediately — that discipline feeds every pipeline downstream. Continue in **Pipelines, Build, Test & Artifacts**.

🎓 **EXAM** CI is **discipline plus machinery**: integrate daily, verify every merge automatically, fix a red main first. PR gates, merge queues and required checks enforce it; trunk discipline, flags and a releasable main sustain it. **Practice versus pipeline**: habits versus the automation that enforces them.

Threads: [Trunk Discipline](#)



Quick Revision — Terms Table

Pipeline	The automated workflow from commit to production.
Stage	A logical phase: build, test, scan, deploy.
Job	A unit of executable work inside a stage.
Step	A single operation inside a job.
Runner	The machine executing jobs; agent in Jenkins.
Artifact	Versioned build output: image, JAR, binary.
Digest	Content hash pinning the exact artifact (sha-9f3a).
Registry	Storage and distribution for artifacts.
Promotion	Approving the same artifact for the next environment.
Gate / Check	A required verification before proceeding.

Merge queue	Serializes merges onto latest green main.
Trunk-based	Tiny branches merged daily; flags hide unfinished work.
Branch	An independent line of work forked from main.
Pull request	Merge proposal coupling human review with CI checks.
Trigger	The event starting a pipeline: push, PR, schedule, manual.
Staging	Prod-like rehearsal; deploys automatic.
Production	Real users; deploys gated or automated.
Approval	The human or policy decision releasing to production.
Continuous Delivery	Every change releasable; deployment is a decision.
Continuous Deployment	Every passing change ships by itself.

KEY IDEA Cover the right column and define each term aloud. Any stumble goes back to its chapter.

Threads: [Controllers & Agents](#) · [Artifact Digest](#) · [Approval Gates](#)



Concept Questions — Distilled Answers

Q1 Delivery vs Deployment?

Delivery keeps software production-ready behind an approval; Deployment releases validated changes automatically.

Q2 Why must staging and production share one artifact digest?

Otherwise you tested one binary and shipped another — verification is void.

Q3 Explain the PR as a quality gate.

Human review for design plus automated checks for correctness plus merge policy: green and approved equals mergeable.

Q4 Name the trigger types.

Push, pull request, schedule, manual dispatch, tag or release.

Q5 CI practice vs CI pipeline?

Discipline — merge small, verify, fix first — versus machinery: the pipeline that enforces it.

Q6 What causes merge hell, and what fixes it?

Long-lived divergent branches; trunk-based discipline with tiny daily merges and flags.

Q7 A green build passed. Does the app work in production?

Not necessarily — it means only the checks inside that pipeline passed.

Q8 Trunk-based vs Git Flow?

Trunk: branch to PR to main to deploy, for CD teams. Git Flow: develop plus release branches, for versioned releases.

Q9 ★ Tell me about a checkout that broke main. What did you change?

STAR story: red main on a Friday deploy — stopped the line, reverted first, then landed a required check so the class of breakage can never merge again.

Threads: [Approval Gates](#) · [Trunk Discipline](#) · [Artifact Digest](#)

Command Cheat Sheet

Git

```
git checkout -b feat/x
```

short branch, merge daily

```
git push -u origin feat/x
```

publish + trigger CI

```
gh pr create --fill
```

PR with CI gate

Docker

```
docker build -t registry/app:SHA .
```

versioned image

```
docker push registry/app:SHA
```

same digest everywhere

NOTE Diagnostic order: red gate first — read the failing check, reproduce locally, fix, re-verify. Never bypass.

Threads: [Artifact Digest](#)

Self-Quiz + Answer Key

Q1 What single question does CI answer?

Q2 Name the six pipeline stages in order.

Q3 What makes an artifact promotable?

Q4 Green plus approved equals...

Q5 What does a merge queue kill?

Q6 Flags hide unfinished work instead of...

Q7 Who fixes a red main, and when?

Q8 Delivery vs Deployment in one line each?

Answer key

1. Does this change work?
2. Source, build, test, artifact, release, deploy.
3. An immutable digest stored versioned in a registry.
4. Mergeable.
5. The green-on-branch-red-on-main race.
6. Long branches.
7. Whoever broke it — first, before anything else.
8. Delivery: releasable behind a decision. Deployment: shipped automatically.

FINAL CHECK Eight for eight from memory, twice in a row — the foundations are yours. Open **Pipelines, Build, Test & Artifacts** next.

Threads: [Artifact Digest](#) · [Trunk Discipline](#)